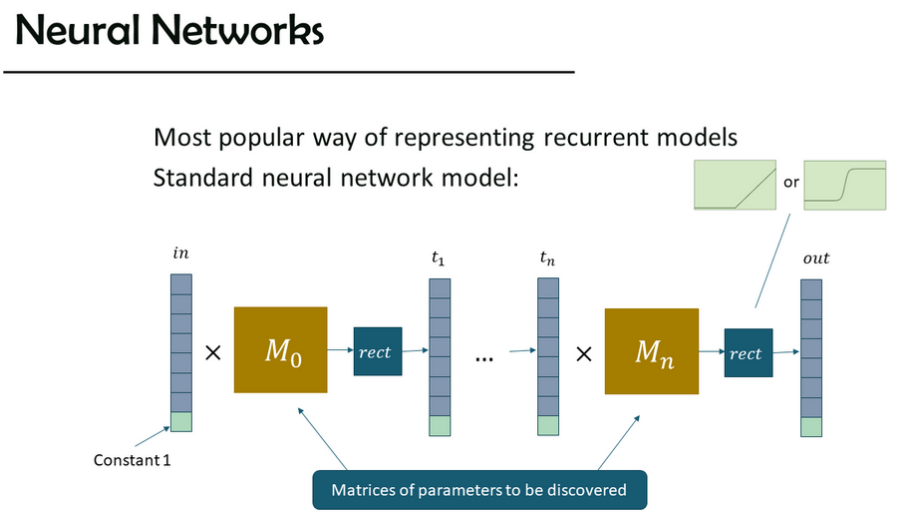
Lecture 10

Basics of Language Modeling for Code

Unit 1 of this course focused on classical, search based approaches to program synthesis. While these approaches have been very successful in specialized domains, scaling them up to general-purpose programming has proven very challenging. Partially as a result of that, and partially as a result of the abundance of code freely available on the web, recent years have seen a surge interest in a completely different approach to synthesis: learning (large) language models for code. In this lecture, we will begin our exploration of these techniques. While we will not be able to cover all the details, we will be able to build a solid intution for how these ideas, which originated in the world of natural language processing, have become so popular for program synthesis.

Throughout this unit, we will be assuming a string representation of programs. As we saw in Lecture 2, there are some tradeoffs to this representation compared with the tree representation we have been using so far, but the overwhelming advantages of the string representation for training large language models has made them the standard representation for neural approaches to program synthesis.

10.1 Multi-layer perceptrons



The basic building block for the learning approaches we are going to be ex-

ploring in this unit is the single-layer perceptron, which takes as input a vector x, multiplies it by a matrix M, and then applies a non-linear function to every element to the output vector. The non-linear functions are often the function $max(in, 0)$, known as a rectified linear unit or RELU, or S shaped functions such as $tanh$ or sigmoid.

Each entry in the Matrix is a tunable parameter; by changing these parameters, we can change the behavior of the function. These perceptrons can be stacked into multi-layer perceptrons, which can be used to approximate arbitrary functions. There is in fact a classical theorem that proves that 3 layers is all that is needed to approximate any function, but in practice, deeper networks do a better job at approximating more complex functions. These multi-layer perceptrons can be trained to match observed data by optimizing the parameters through gradient descent to minimize an error function for a given output.

10.2 Distributions over programs, lessons from NLP

As we saw in Lecture 2, the program synthesis problem can be framed in probabilistic terms as finding the program that maximizes the posterior probability given some evidence. In the case of inductive synthesis, the evidence took the form of input-output examples, but the learning techniques we will be studying in this lecture will allow us to directly learn conditional distributions with richer forms of evidence, such as natural language specifications. The name of the game for the rest of the lecture is going to be on how to train neural networks to represent these conditional distributions.

The central challenge we have to overcome in order to do any kind of language modeling is that our basic building blocks map input vectors to output vectors, but our inputs and outputs consist of sequences of tokens, so we have to be able to represent distributions over sequences of arbitrary length using building blocks that operate on fixed-sized vectors.

The simplest way to represent a sequence using building blocks with fixed-size inputs is the **n-gram model**. In this model, we make an assumption that the probability of a token depends only on a small window of preceding tokens.

n-gram models

The big brown bear scares the children with its roar

$P(scares \mid bear, brown)$

Probability of a word depends on the previous n words

Represented with a table: $P(w_i \mid w_{i-1}, w_{i-2}, \ldots, w_{i-n})$

Bigger n makes more accurate, but also more difficult to learn, requires much bigger table

Downsides

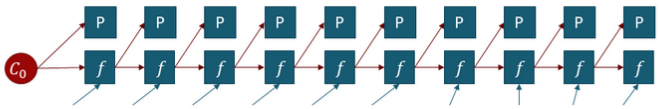
- some words require more context than others
- some words carry very little information

Recurrent models

Key idea: let the system figure out how to construct its own context

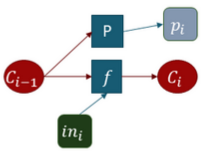
Now need to learn two interrelated functions

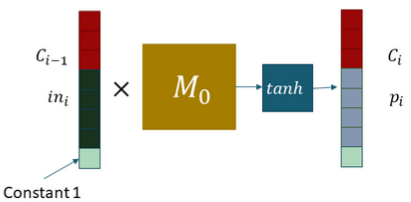
- $P(word_i \mid context_{i-1})$
- $context_i = f(word_i, context_{i-1})$



The big brown bear scares the children with its roar

Simple recurrent neural network





So if a sentence is represented as a sequence of words $[w_0, w_1, \ldots, w_k]$. Then an n-gram model assumes that for every word, the probability of that word given all the preceding words depends only on the previous n-words.

$$P(w_i|w_{i-1}, w_{i-2}, \ldots w_0) = P(w_i|w_{i-1}, w_{i-2}, \ldots w_{i-n})$$

For small n, n-gram models are easy to learn; in fact, if your n is very small, you don't even need a neural network; a table will suffice. But they have very limited expressive power. For example, consider the sentence

The big brown bear scares the children with its roar

In the sentence above, the word 'bear' is strongly determined by the previous two words, but in order to predict the word 'roar' with any accuracy, you would have to look back at least six words to know that you are talking about a bear.

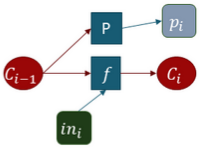
Recurrent models. One way to get around this problem is to have a formalism that can learn its own context, instead of relying on a short window of previous tokens.

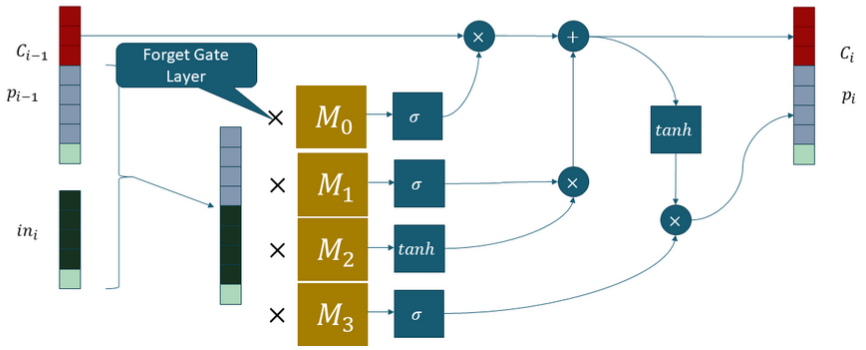
$$P(word_i|context_{i-1})context_i = f(word_i, context_{i-1})$$

In this case, the idea is that the context at every step is a function of the current word and the context up to that point, and the distribution over words at every step depends exclusively on this computed context.

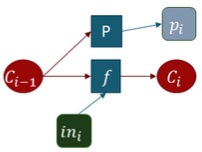
The simplest way to instantiate the above model with neural networks is the *recurrent neural network*. In this model, a vector is used to represent both the context and the current word. This vector is passed through a neural network to compute a new vector representing the new context, and a distribution over words.

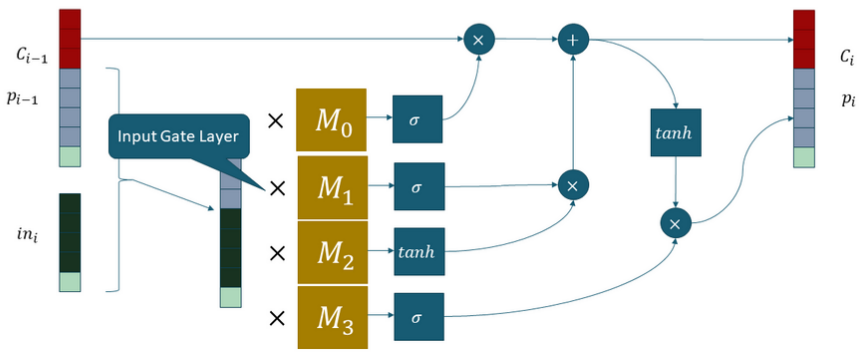
LSTM

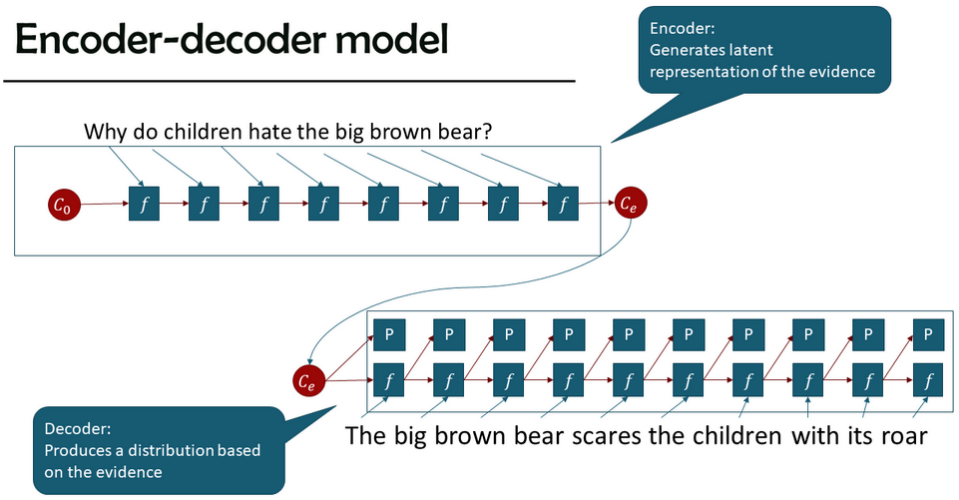

P
p_i
C_{i-1}
f
C_i
in_i

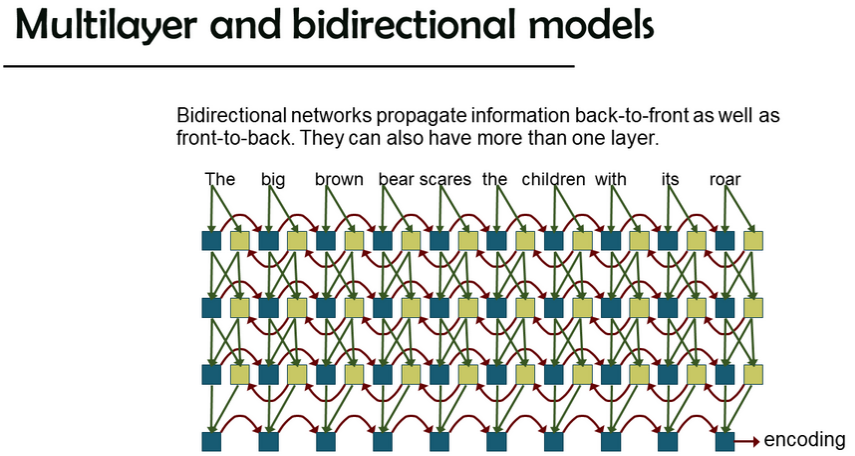

C_{i-1}
p_{i-1}
in_i
Forget Gate Layer
M_0
M_1
M_2
M_3
σ
σ
$tanh$
σ
$tanh$
C_i
p_i

LSTM


P
p_i
C_{i-1}
f
C_i
in_i


C_{i-1}
p_{i-1}
in_i
Input Gate Layer
M_0
M_1
M_2
M_3
σ
σ
$tanh$
σ
$tanh$
C_i
p_i





The simple recurrent neural network has been superseded by more sophisticated models that are easier to train. One model that became popular around 2014 is the Long Short Term Memory (LSTM) [1], illustrated in the figure. The basic principle is the same one; the difference is in the way the context is propagated. Instead of simply pushing the context through the full network, the context is incrementally modified by first scaling it down by the result of a *forget gate*, and then additively updated by the result of an *input layer gate*. As a result, gradients can propagate better past multiple stages of the computation, allowing the networks to be trained with longer sequences of text.

10.3 Conditional distributions

One of the strengths of using neural networks for sampling distributions is that it is easy to generalize to conditional distributions, where the distribution depends on some evidence or observation. The evidence can be a set of input/output examples, or it can be something less precise, such as natural-language text.

$$P(program|evidence)$$

One approach to encoding these distributions is to use an encoder-decoder model, where a recurrent network is used to encode the evidence into a vector, and then the vector is used as the initial context for a decoder network. Such sequence-to-sequence models were popular for language translation starting in 2014 with the work of Sutskever, Vinyals and Le [2], and are broadly applicable to other tasks such as question answering, or in our context, program synthesis. There is a broad literature of similar models that vary in how the encoding and decoding is done. For example, a common strategy is to encode the sequence using multi-layer bidirectional models that propagate information about the sequence both front-to-back and back-to-front.

Back in 2016, we published some early work using sequence to sequence models to generate programs [3]. For that work, we designed a model to synthesize a line of code given the lines of code before and after, and we trained it on correct solutions to programming assignments from a MOOC. The goal was to use this model to automatically make corrections to student submissions for the same assignments.

10.4 Sequence-to-sequence models with attention

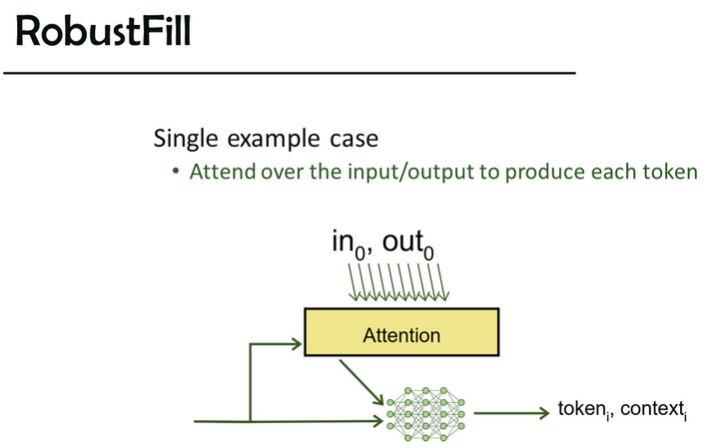
The basic encoder-decoder model shown in the figure has major limitations that derive from the fact that all the evidence about the entire sentence must be encoded into a single vector. For this reason, they were quickly extended with *Attention*-based models, first introduced in this context by Bahdanau, Cho and Bengio [4].

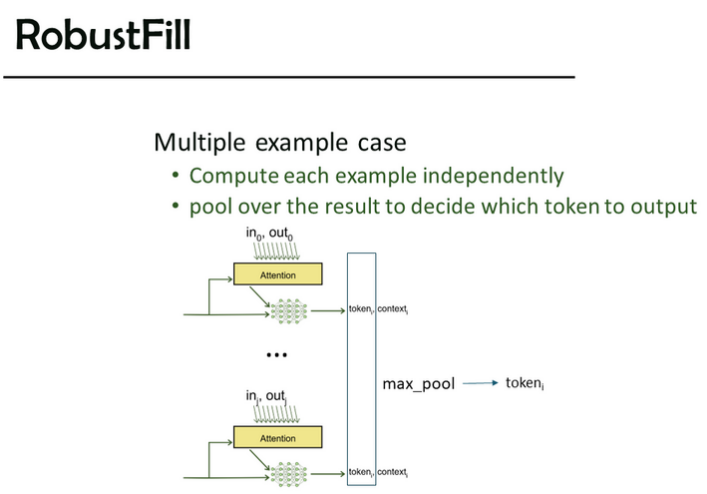
The key insight behind attention is that there is some degree of locality, where different tokens of the output depend primarily on small subsets of tokens from the input, so instead of encoding the entire input into a single fixed vector, the attention mechanism allows each output token to pay attention to a different subset of input tokens. This means that it is no longer necessary to pack all the information from the input sequence into a single vector; instead, the network dynamically selects the relevant information for each output token. The details of this mechanism are illustrated in the figure.

The input sequence is encoded by a bidirectional recurrent network, but instead of aggregating the values into a single vector, the values from each word

in the input are passed to the attention mechanism. For each token in the output, the network computes a vector called a *query* that is combined with the vector from each input token to produce a weight for each intermediate vector from the input. The output of the attention mechanism is a vector computed as a weighted sum for each token in the output. This means that each step in the output will get a different weighted sum corresponding to the input tokens that are more or less relevant for computing that output.

One of the first systems to use attention for program synthesis was RobustFill [5]. This paper proposed and compared several different architectures for encoding inductive synthesis problems for the domain of text editing. Specifically, the goal of this paper is to produce a neural-guided synthesis version of FlashFill. One of the important observations of this paper was that an instance of a synthesis problem takes as input a set of input/output pairs, each of which involves two strings (the input and the output) of unbounded length, so there are actually some design decisions involved in how to handle that. The most successful approach used by that paper was to use attention within an individual input/output pair, but then aggregate over the distributions proposed from each of the examples.





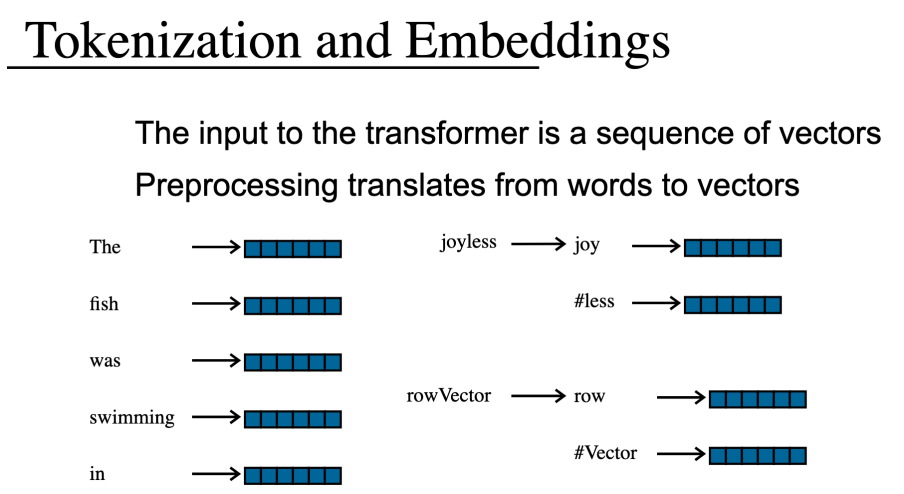
This choice of using attention within an example and pooling accross example works well in this domain because there is actually a fair bit of locality between the programs and the input/output examples. For example, consider the following example and program.

```
        in: "Armando Solar-Lezama"
        out: "A. Solar-Lezama"

        Program: Concat(SubString(in, Pos("", Word), Pos(Char,"")), ". ",
    SubString(in, Pos(" ",Word), Pos("", End));
```

The program consists of three parts, an expression that extracts the first initial, concatenated with a constant, and then with an expression that extracts everything after the first space. Notice, however, that each of these parts can be inferred independently by focusing on particular regions of the input and output strings. By contrast, if you were trying to synthesize complicated arithmetic expressions, for example, there would not be so much locality to exploit, so the benefits of using attention would be significantly reduced.

10.5 Tokenization



Before continuing with Transformers, there is one detail that we have more or less swept under the rug: how does the input text get turned into the actual input sequence fed to the model. The models operate on sequences of vectors, but our input is best interpreted as either a sequence of words or a sequence of characters, but both have significant shortcomings. On the one hand, individual characters have too little semantic meaning, and even short blocks of text could turn into very long vector sequences if we treat each character as its own vector. Words, however, are not ideal either; for one, the space of possible words is quite large and not necessarily closed; people often coin new words by combining or modifying existing words, and we can often make inferences about the meaning of new words from their parts.

In modern language models, each vector in the input represents a *token*. Tokens can represent full words, but complex words are often broken into multiple tokens. Input text is broken into tokens by a tokenizer before being fed into a model.

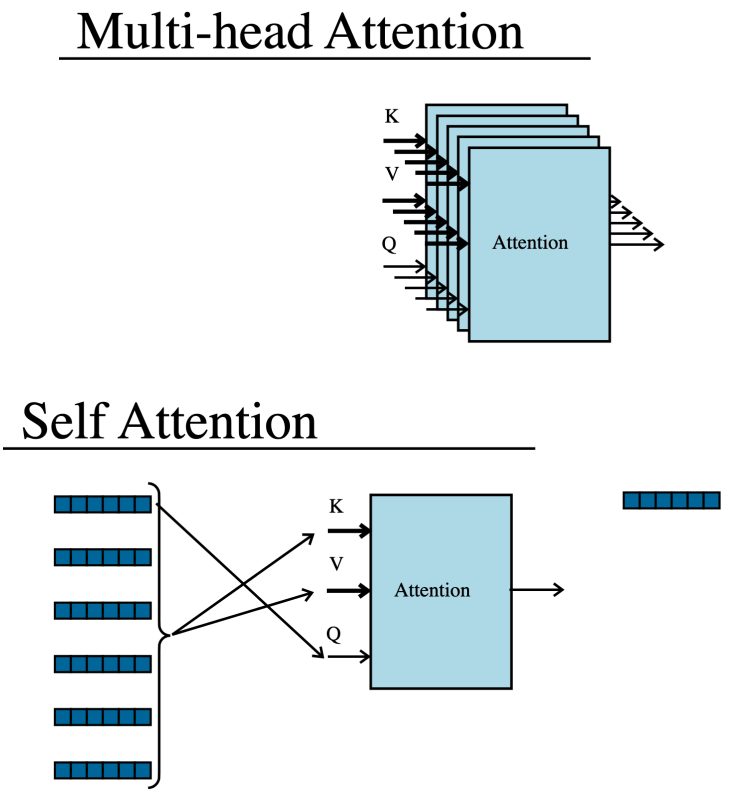
Tokenization is especially tricky for code for several reasons. First, variable names and function names are often compound words which carry a lot of semantic information, so it is important for the tokenizer to capture the individual words that make up a variable name. Code also often manipulates string constants in ways that require a detailed character-by-character understanding of the string. Even GPT-4, for example, often had trouble interpreting short bits of code that involved, for example, extracting a sub-string from a string.

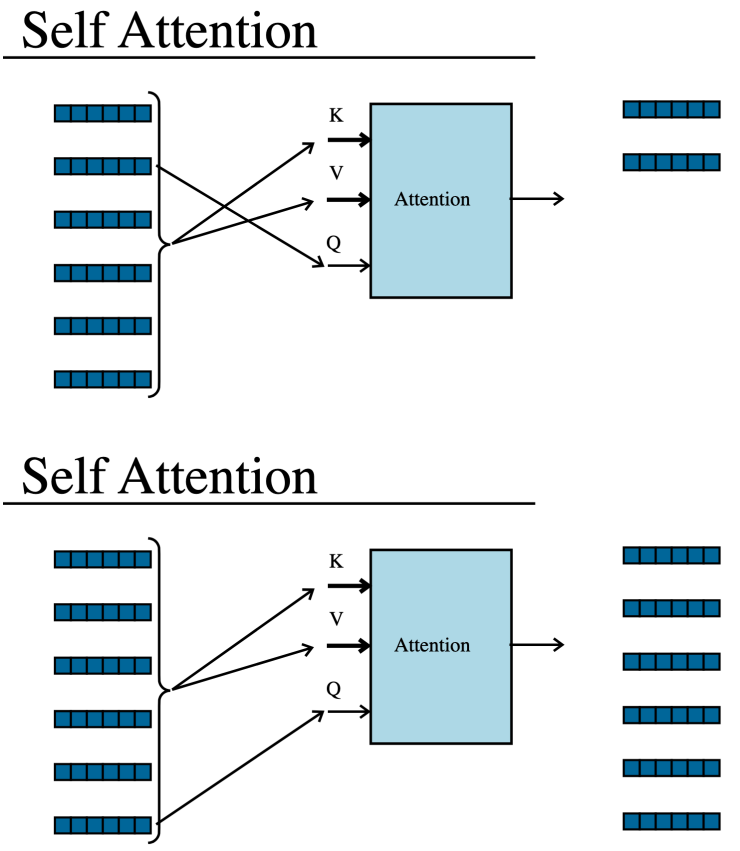
10.6 Transformers: Preliminaries

The basic use of attention described earlier still suffers from a couple of important problems. First, while we are now using the attention mechanism to avoid having to encode the entire *input* as a single vector, the decoder is still a recurrent neural network that propagates information step by step, so that on a long output, we will still have the same problem of having to encode all relevant information about the output in a fixed size vector. Additinally, the encoder itself still relies on a recurrent network to compute the individual vectors that will be passed to the attention mechanism.

In an aptly titled paper in 2017 [6], Vaswani et al. argued that all of those recurrent networks could be replaced with attention mechanisms in order to get a more scalable network. Their proposed architecture is called a Transformer and it forms the basis of all large-scale language models from the likes of OpenAI and Anthropic.

10.6.1 A second look at attention





To better understand transformers, we need a better understanding of the attention mechanism itself. One way to understand the attention mechanism is as a key-value store. A key value store gets as input a database of key-value pairs and a query, and its goal is to find the key that is closest to the query and return the value associated with that key. Now, suppose I have a collection of keys $K = [k_0, k_1, \ldots k_i]$, and a set of values $V = [v_0, v_1, \ldots, v_i]$. Now, if every key is a column vector, I can represent the collection as a matrix K where each column is a key. Multiplying a query vector q times K^T will yield a vector whose i^{th} entry corresponds to the dot product of q and k_i, so higher values will correspond to a closer match between key and query. We can apply a softmax to that vector so that the smallest values will get closer to zero and the largest values will get closer to one. Multiplying that vector will provide us with a linear combination of the values whose keys were closest to q.

$$V \cdot softmax(K^T \cdot q)$$

The attention mechanism does exactly this with one small difference: The

vector q, the matrix K and the vector V are each multiplied by a separate weight matrix.

$$Attention(q, K, V) = W_v \cdot V \cdot softmax(W_k \cdot K^T \cdot W_q \cdot q)$$

In the attention network presented earlier, the query corresponded to the query vector that came from the output RNN (which encodes the output produced so far), while the Keys and Values both corresponded to the vectors coming from the input RNN.

Multi-headed attention In some cases, the same Query, Keys and Values can be passed through multiple attention functions in parallel, each with its own weights to produce a collection of vectors.

$$MHAttention(q, K, V) = [Attention_0(q, K, V), Attention_1(q, K, V), \ldots, Attention_k(q, K, V)]$$

This is called multi-headed attention.

Self-attention We have already seen an example where the Keys and Values are actually the same collection of vectors. We can use the same vectors as Queries as well; this is known as *self-attention*. Given a collection of vectors $H = [h_0, \ldots, h_k]$, we can compute

$$MHAttention(h_i, H, H)$$

The effect of self attention is to allow each vector in H to be translated into a matrix that incorporates information about all other vectors in H. Why would we want this? Well, in the language translation problem, for example, you often want to encode each word into a vector that represents the meaning of that word, but some words can have multiple meanings, so you need some context to decide which meaning to choose. This means that a good encoding should not just map each word to a vector independently but should pay attention to all the other words in the context. So for example, if you have the sentence "The sheep were happy to be back in their pen.", an encoding of the word *pen* needs to pay attention to the other words in the sentence, especially the word *sheep*, because that determines how *pen* should be encoded. In particular, we want an encoding that will distinguish from the use of *pen* in the sentence "For this writer, no other posession was more valuable than her pen."

10.7 Transformers: How they work

With these concepts defined, we can now describe the key ideas behind the original transformer architecture.

Self-Attention for the input. The first idea is that we are going to compute a set of vectors for each token in the input, but we are going to do this using

self-attention instead of the bidirectional recurrent network that was used by the earlier Bahdanau paper.

Using attention has the advantage of avoiding long chains of dependencies and is therefore more scalable, but it introduces a small complexity: The attention mechanism is insensitive to the order of the Key/Value vectors. So when replacing the RNN with the self-attention mechanism, we need a way to incorporate position information into the process so that the network can take into account the actual order of words and their proximity to each other when computing their vectors. This is generally done by adding to the vector for each word a vector that encodes position information.

Self-Attention for the output. The second idea is that instead of using a recurrent neural network to track the output computed so far, the Transformer will use another multi-headed self-attention module to encode the output so far. This will also require encoding position information straight into the vectors using the same mechanism as before.

Cross-Attention for combining inputs and current output together. The transformer will then use another attention module to combine together the vectors from the input with the vectors from the output so far. In particular, the vectors coming from the input will work as Keys and Values, while the vectors coming from the so-far-computed output are used as queries. This is typically called *cross-attention*.

Additional machinery. These are the key ideas behind transformer networks, but the detailed architecture has some additional complexity. For example, in the encoder, there are multiple layers of self-attention stacked on top of each other with pass-through layers and small feed forward layers in between them. In the decoder, there are also multiple layers that combine the output so far with the Keys and Values computed from the input, and these are also interleved with pass through layers and feed forward layers, but the ideas outlined here are the main ideas that make the transformer unique. **Position encodings** One thing you may have noticed about the descriptions so far is that the attention mechanism is oblivious to the order in which tokens are given. In practice, though, the actual order of the tokens really matters, especially when we are dealing with code. In modern transformers, the order of input tokens is captured by incorporating position information directly into the input vectors representing each token using a specialized position encoding.

Causal decoder-only models. While the original transformer architecture was designed to be an encoder-decoder model that can be used with a range of different objectives, the community quickly realized that it was typically both easier to train and more effective to build models that only used the decoder part of the architecture. What this means is that the input is simply treated as hard-coding the first few tokens of the output; at each step, the model then uses self-attention to compute the distribution over the next token based only

on the preceding tokens (this is known as *causal* attention). Thus, unlike in the encoder-decoder setting, there is no cross-attention mechanism: it is all self-attention, all the way down. Encoder-decoder models were popular in early academic work on transformers, but are now predominantly used in specialized applications such as vector search and sentiment analysis (where there is a clear distinction between the input and output space). At the time of writing, all the most popular large-scale language models are causal decoder-only models; including GPT-4, the o-series models, Gemini, Claude, and Llama.

10.8 Autoregressive training

Early efforts at using language models to generate code relied on specialized training for specific programming tasks. For example, in a previously mentioned paper [3], we explored the use of language models to correct programming assignments. In this case, we trained the models with a large number of correct solutions to the programming assignment, and the model was trained to predict a line of code by taking as input the adjacent lines of code. This led to a model that was very effective at its specific task; given a buggy program, we could compute the probability of each token in each line of code given the lines of code around it, and if the current line was deemed to have low probability, replacing it with a higher probability line would often fix the problem, but such a model was only useful for those assignments in the corpus and it had to be trained anew for new assignments.

As language models became more capable and able to use longer inputs, it started to become possible to train them on code collected from the internet, but they still relied on specialized training for particular tasks. For example, in a system called Bayou, the system was explicitly trained to map from an initial set of API calls to a program that used those API calls.

In recent years, many of these specialized approaches to training language models have been replaced by a more general approach, where a single large model is pre-trained on a large corpus of code and natural language, and then this model is used for a variety of tasks. In order to do this, this approach needs a very simple training objective that is not specific to any particular task. The most common training objective used for this purpose is *autoregressive training*, where the model is trained to predict the next token in a sequence given the previous tokens.

10.9 What will come after transformers?

The broad successes of current-generation LLMs begs the question of how much of the success is due to the architecture itself, and how much is due to the scale of the models. Indeed, while the transformer architecture is undoubtedly

powerful (and lends itself very nicely to parallelization on modern hardware like GPUs and TPUs), it is not without its limitations. One particular such limitation that has been discussed heavily in the literature is that self-attention involves $O(N^2)$ operations to produce a sequence of length N. This is in stark contrast to the $O(N)$ operations required by recurrent networks. In the context of program synthesis, where we would in an ideal world be able to condition on entire codebases, this is a significant limitation.

The community has come up with several different approaches to address this scaling challenge. Early work [7] considered simply removing the softmax operation from the attention mechanism, yielding:

$$Attention(q, K, V) = W_v \cdot V \cdot W_k \cdot K^T \cdot W_q \cdot q$$

which, through some clever factorizing of the multiplication, yields a linear-time attention mechanism. Unfortunately, this approach did not appear to work well in practice; the non-linearity introduced by the softmax operation appeared to be critical. However, there has been some success in training commerical-scale models that predominantly use linear attention mechanisms, retaining the softmax only in a few key layers [8].

Other approaches that the literature has explored include scaling LSTMs to longer sequences on modern hardware [9], state-space models such as Mamba [10] [11] and modern convolutional models such as Hyena [12]. Unfortunately, discussing these works in detail is beyond the scope of this course; If you are interested in learning more, I recommend checking out a recent overview by Sieber et al. [13] of how these formalisms compare to one another.

Either way, it is safe to say that the jury is still out on what will come after transformers.

References

[1] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9:1735–80, 12 1997. doi: 10.1162/neco.1997.9.8.1735.

[2] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *Proceedings of the 27th International Conference on Neural Information Processing Systems - Volume 2*, NIPS'14, page 3104–3112, Cambridge, MA, USA, 2014. MIT Press.

[3] Yewen Pu, Karthik Narasimhan, Armando Solar-Lezama, and Regina Barzilay. sk_p: a neural program corrector for moocs. In Eelco Visser, editor, *Companion Proceedings of the 2016 ACM SIGPLAN International Conference on Systems, Programming, Languages and Applications: Software for Humanity, SPLASH 2016, Amsterdam, Netherlands, October 30 - November 4, 2016*, pages 39–40. ACM, 2016. doi: 10.1145/2984043.2989222. URL https://arxiv.org/abs/1607.02902.

[4] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. URL http://arxiv.org/abs/1409.0473.

[5] Jacob Devlin, Jonathan Uesato, Surya Bhupatiraju, Rishabh Singh, Abdel rahman Mohamed, and Pushmeet Kohli. RobustFill: Neural program learning under noisy I/O. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 990–998, International Convention Centre, Sydney, Australia, 06–11 Aug 2017. PMLR. URL http://proceedings.mlr.press/v70/devlin17a.html.

[6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

[7] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: fast autoregressive transformers with lin-

ear attention. In *Proceedings of the 37th International Conference on Machine Learning*, ICML'20, 2020.

[8] MiniMax. MiniMax-01: The official repo of MiniMax-Text-01 and MiniMax-VL-01, large-language-model & vision-language-model based on Linear Attention. https://github.com/MiniMax-AI/MiniMax-01, 2025. Accessed on 2025-06-24.

[9] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xlstm: Extended long short-term memory. In *Thirty-eighth Conference on Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2405.04517.

[10] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[11] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning (ICML)*, 2024.

[12] Michael Poli, Stefano Massaroli, Eric Nguyen, Daniel Y. Fu, Tri Dao, Stephen Baccus, Yoshua Bengio, Stefano Ermon, and Christopher Ré. Hyena hierarchy: towards larger convolutional language models. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23, 2023.

[13] Jerome Sieber, Carmen Amo Alonso, Alexandre Didier, Melanie N. Zeilinger, and Antonio Orvieto. Understanding the differences in foundation models: attention, state space models, and recurrent neural networks. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, 2024.